Learning from Big Data: Tutorial 1.1 (Solutions)

LfBD Team, 2026

September 2026

Introduction

This tutorial focuses on applying Natural Language Processing (NLP) techniques for supervised learning. We begin by using the Naive Bayes Classifier (NBC) on review data to determine both the topic and sentiment of a review. Next, we apply AFINN, a lexicon-based tool, to perform the same sentiment analysis task. Additionally, we will cover how to evaluate model performance using a confusion matrix, explaining how it visualizes predictions, can highlight incorrect classifications, and supports the calculation of key performance metrics used to evaluate our models.

1. Loading libraries

Before starting the tutorial, make sure you have all the required libraries properly installed. Simply run this chunk of code below.

```
# Required packages. P_load ensures these will be installed and loaded.
if (!require("pacman")) install.packages("pacman")
pacman::p_load(tm, nnet, dplyr, tidyr, ggplot2, reshape2,latex2exp, syuzhet, caret, png, knitr)
```

2. Load the reviews and prepare the data

```
setwd("C:/Users/josef/Desktop/Learning-from-Big-Data-main/Learning-from-Big-Data-main/tutorials/input")
# Load the review data. Note that we are now using the fileEncoding parameter when
# calling read.csv() - this helps reading the review text correctly for further
# processing (by correctly interpreting the non-ASCII symbols)
# The ISO-8859-1 encoding is used to represent the first 256 uncicode characters
reviews_raw <- read.csv('Reviews_tiny.csv', fileEncoding="ISO-8859-1")

reviews_raw <- reviews_raw %>%
  select(movie_name,review_code,    reviewer,   review_date, num_eval,
         prob_sentiment,words_in_lexicon_sentiment_and_review, ratio_helpful,
         raters,
         prob_storyline,    prob_acting,    prob_sound_visual,
         full_text, processed_text,
         release_date,  first_week_box_office,MPAA, studio, num_theaters )
```

Texts labelled with content or sentiment are subsequently used to compute word likelihoods. The code chunk below loads training data from three content lexicons related to storyline, acting, and visual aspects. Sentences are assigned to topics like acting, storyline, or visuals for topic classification using NBC. Furthermore, content/sentiment likelihood data is loaded. These are from movie reviews labelled as positive or negative used in our sentiment analysis. Finally, the parameters capturing the priors for our NBC models are defined

```
# training data
dictionary_storyline <- read.csv2("storyline_33k.txt")
dictionary_acting    <- read.csv2("acting_33k.txt")
dictionary_visual    <- read.csv2("visual_33k.txt")

# TODO: Compute the word likelihoods from 3 content dictionaries (i.e., your training data).
# Here, I load a list of 100 words with fake content likelihoods and a list with
# 100 fake sentiment likelihoods. These are just examples. These 100-word lists
# are not to be used in your assignment. In your assignment, you are expected to
# compute the content likelihoods for all the words in the training data yourself.
likelihoods <- read.csv("example_100_fake_likelihood_topic.csv")
```

SOLUTION. A likelihood is a relative frequency: P(word | topic) = how often the word occurs

in that topic's training text, divided by the total number of words of that topic. So we count, and then we divide.

The words of the three dictionaries. tolower() makes "Hero" and "hero" the same word

```
words_storyline <- tolower(dictionary_storyline[ ,1])
words_acting    <- tolower(dictionary_acting[ ,1])
words_visual    <- tolower(dictionary_visual[ ,1])

# Counting the same word over and over inside the loop would be slow, so we count each
# dictionary once. table() returns, for every word, the number of times it occurs
count_storyline <- table(words_storyline)
count_acting    <- table(words_acting)
count_visual    <- table(words_visual)
```

The vocabulary: every word occurring in at least one of the three dictionaries. We take all

three, not just one, because a word that only shows up under storyline still needs a (small) likelihood under acting and visual

```
unique_words <- sort(unique(c(words_storyline, words_acting, words_visual)))

# The table we fill in: one row per word, one column per topic
results_matrix <- data.frame(words     = unique_words,
                             storyline = 0,
                             acting    = 0,
                             visual    = 0,
                             stringsAsFactors = FALSE)

for (i in 1:nrow(results_matrix)) {

  word <- results_matrix$words[i]

  # look the word up in each count; an absent word gives NA, meaning it occurred zero times
  s_count <- count_storyline[word]
  a_count <- count_acting[word]
  v_count <- count_visual[word]

  if (is.na(s_count)) s_count <- 0
  if (is.na(a_count)) a_count <- 0
  if (is.na(v_count)) v_count <- 0
```

```
  # add one to every count (Laplace smoothing): without it a word missing from the acting
  # dictionary would get a likelihood of exactly zero, and that single word would push the
  # posterior of acting to zero regardless of what the rest of the review says
  results_matrix$storyline[i] <- s_count + 1
  results_matrix$acting[i]    <- a_count + 1
  results_matrix$visual[i]    <- v_count + 1
}
```

Turn the counts into probabilities by dividing every count by its COLUMN total, i.e. by the

total number of words of that topic. Every column then sums to one, because a column is the distribution of that one topic over the whole vocabulary: that is what P(word | topic) means. (Dividing by the row total instead would look tidier - the three numbers of a word would sum to one - but it is a different quantity, and it drops the correction for one dictionary containing more training text than another.)

```
likelihoods <- results_matrix
likelihoods$storyline <- results_matrix$storyline / sum(results_matrix$storyline)
likelihoods$acting    <- results_matrix$acting    / sum(results_matrix$acting)
likelihoods$visual    <- results_matrix$visual    / sum(results_matrix$visual)

# Check: each column sums to one, and the words that take most of their probability from one
# topic (counting only words seen at least 10 times) should look like that topic
colSums(likelihoods[ ,2:4])

for (topic in c("storyline", "acting", "visual")) {
  share  <- likelihoods[[topic]] / (likelihoods$storyline + likelihoods$acting + likelihoods$visual)
  enough <- results_matrix$storyline + results_matrix$acting + results_matrix$visual >= 10
  cat(topic, ":", head(likelihoods$words[enough][order(-share[enough])], 8), "\n")
}

## storyline : moral opponent plot hero story revelation heros stories
## acting : rehearsal exercises tension class relaxation objective givens concentration
## visual : digital animation data previs matte effects model shots

# TODO: Locate a list of sentiment words that fits your research question.
# This is available from the literature. For example, you may want to look at
#just positive and negative (hence two dimensions), or you may want to look at
# other sentiment dimensions, such as specific emotions (excitement, fear, etc.).
# The list of 100 words with fake likelihoods for sentiment used below is not to be used.
likelihoods_sentim <- read.csv2("example_100 fake_likelihood_sentiment.csv",header=TRUE,
                                sep=",", quote=" \"",dec=".",fill=FALSE)

# SOLUTION. Exactly the same exercise as above, only now the training data are the two
# sentiment word lists that come with the course: a list of positive and a list of negative
# words (from the literature, so no need to build them ourselves)
positive_words <- tolower(read.csv("positive.csv")$x)
negative_words <- tolower(read.csv("negative.csv")$x)

# As with the content dictionaries, count each list once instead of searching it inside the
# loop. These are word lists, so a count is 1 if the word is on the list and 0 if it is not
count_positive <- table(positive_words)
count_negative <- table(negative_words)

# The sentiment vocabulary: every word that appears on either list
sentiment_words <- sort(unique(c(positive_words, negative_words)))
```

```
sentiment_matrix <- data.frame(words          = sentiment_words,
                               pos_likelihood = 0,
                               neg_likelihood = 0,
                               stringsAsFactors = FALSE)

for (i in 1:nrow(sentiment_matrix)) {

  word <- sentiment_matrix$words[i]

  # look the word up in each count; an absent word gives NA, meaning it occurred zero times
  p_count <- count_positive[word]
  n_count <- count_negative[word]

  if (is.na(p_count)) p_count <- 0
  if (is.na(n_count)) n_count <- 0

  sentiment_matrix$pos_likelihood[i] <- p_count + 1   # Laplace smoothing, as above
  sentiment_matrix$neg_likelihood[i] <- n_count + 1
}

# Again divide by the COLUMN total, so that each of the two columns sums to one
likelihoods_sentim <- sentiment_matrix
likelihoods_sentim$pos_likelihood <- sentiment_matrix$pos_likelihood / sum(sentiment_matrix$pos_likeliho
likelihoods_sentim$neg_likelihood <- sentiment_matrix$neg_likelihood / sum(sentiment_matrix$neg_likeliho

# Check: both columns sum to one, and a word from the positive list is about twice as likely
# under positive as under negative (a word on neither list is not in this table at all)
colSums(likelihoods_sentim[ ,2:3])
likelihoods_sentim[likelihoods_sentim$words %in% c("hero", "awful"), ]

##      words pos_likelihood neg_likelihood
## 338  awful   0.0001900057   0.0003551767
## 1681  hero   0.0003800114   0.0001775884
```

Note how coarse this is: every positive word carries exactly the same weight, because a word

list says only whether a word belongs to a sentiment, not how strongly. If you want stronger evidence for "superb" than for "nice", estimate the likelihoods on labelled training reviews (or use a lexicon that comes with valence scores, e.g. AFINN).

These lexicons are used as (a dictionary of) words that are associated with our content topics/sentiment

```
lexicon_content   <- as.character(likelihoods[ ,1])
lexicon_sentiment <- as.character(likelihoods_sentim$words)

# Setting our prior parameters
# We set out priors for the topics to 1/3 each because we have three topics
# (i.e. storyline, acting, and visual). Similarly, we set the priors for
# sentiment to 1/2 each becausse we have two sentiments (positive/negative)
prior_topic <- 1/3
prior_sent  <- 1/2

total_reviews <- nrow(reviews_raw)
```

3. Supervised Learning: Naive Bayes Classifier (NBC)

The Naive Bayes Classifier is a probabilistic model based on Bayes' Theorem used to predict the probability that a given input, in this case reviews, belongs to a particular category. Throughout this tutorial the categories we will be using are sentiment and topics. NBC begins with a prior probability for each class, which represents the initial belief about the likelihood of each category. Then, for every word in the input, the model calculates the likelihood of that word appearing given each class. Using Bayes' rule, it continuously updates the probability for each class as more words are considered. Finally, the class with the highest posterior probability is selected as the predicted category.

Below, the functions Compute_posterior_sentiment and Compute_posterior_content are displayed. These functions apply the Bayes rule to calculate posteriors.

Compute posterior sentiment function

This first function estimates the probability that the review expresses positive or negative sentiment.

```
Compute_posterior_sentiment <- function(prior, corpus_in , dict_words, p_w_given_c,TOT_DIMENSIONS ){

  output <- capture.output(word_matrix <-
                             inspect(DocumentTermMatrix(corpus_in,
                                                        control=list(stemming=FALSE,
                                                                     language = "english",
                                                                     dictionary=as.character(dict_words
```

Check if there are any relevant words in the review, if there are, treat them. if not, use prior

```
  if (sum(word_matrix) == 0) {

    posterior<-prior ; words_ <- c("")

  } else{

    # Positions in word matrix that have words from this review
    word_matrix_indices <- which(word_matrix>0)
    textual_words_vec   <- colnames(word_matrix)[word_matrix_indices]

    # Loop around words found in review
    WR <- length(word_matrix_indices) ;word_matrix_indices_index=1
    for (word_matrix_indices_index in 1: WR){

      word <- colnames(word_matrix)[word_matrix_indices[word_matrix_indices_index]]
      p_w_given_c_index <- which(as.character(p_w_given_c$words) == word)

      # Loop around occurrences | word
      occ_current_word <- 1
      for (occ_current_word in 1: word_matrix[1,word_matrix_indices[word_matrix_indices_index]])
      {
        # initialize variables
        posterior <- c(rep(0, TOT_DIMENSIONS))
        vec_likelihood <- as.numeric(c(p_w_given_c$pos_likelihood[p_w_given_c_index],
                                       p_w_given_c$neg_likelihood[p_w_given_c_index]))

        # positive - this is the first element in the vector
        numerat <- prior[1] * as.numeric(p_w_given_c$pos_likelihood[p_w_given_c_index])
        denomin <- prior %*% vec_likelihood
```

```
        posterior[1] <- numerat / denomin

        # negative - this is the second element in the vector
        numerat <- prior[2] * as.numeric(p_w_given_c$neg_likelihood[p_w_given_c_index])
        denomin <- prior %*% vec_likelihood
        posterior[2] <- numerat / denomin
```

The %*% sign above indicates matrix multiplication, which is beyond the scope of the course

For those interested: https://www.mathsisfun.com/algebra/matrix-multiplying.html

```
        if (sum(posterior)>1.01) {
          ERROR <- TRUE
        }

        prior <- posterior

      } # close loop around occurrences

    } # close loop around words in this review

    words_ <- colnames(word_matrix)[word_matrix_indices]

  } # close if review has no sent words

  return(list(posterior_=posterior, words_=words_) )
}
```

Compute posterior content function

This second function determines the probability that the review pertains to each specific topic.

```
Compute_posterior_content <- function(prior, word_matrix, p_w_given_c , BIGRAM, TOT_DIMENSIONS){
```

Check if there are any relevant words in the review, if there are, treat them.If not, use prior

```
  if (sum(word_matrix) == 0) {

    posterior<-prior

  } else{

    # Positions in word matrix that have words from this review
    word_matrix_indices <- which(word_matrix>0)
    textual_words_vec   <- colnames(word_matrix)[word_matrix_indices]

    # Loop around words found in review
    WR <- length(word_matrix_indices) ;word_matrix_indices_index=1
    for (word_matrix_indices_index in 1: WR) {

      word <- colnames(word_matrix)[word_matrix_indices[word_matrix_indices_index]]
      p_w_given_c_index <- which(as.character(p_w_given_c$words) == word)

      # Loop around occurrences | word
      occ_current_word <- 1
      for (occ_current_word in 1:word_matrix[1,word_matrix_indices[word_matrix_indices_index]])
```

```
      {
        # initialize variables
        posterior <- c(rep(0, TOT_DIMENSIONS))
        vec_likelihood <- as.numeric(c(p_w_given_c$storyline[p_w_given_c_index],
                                       p_w_given_c$acting[p_w_given_c_index],
                                       p_w_given_c$visual[p_w_given_c_index]) )

        # storyline - this is the first element in the vector
        numerat <- prior[1] * as.numeric(p_w_given_c$storyline[p_w_given_c_index])
        denomin <- prior %*% vec_likelihood
        posterior[1] <- numerat / denomin

        # acting - this is the second element in the vector
        numerat <- prior[2] * as.numeric(p_w_given_c$acting[p_w_given_c_index])
        denomin <- prior %*% vec_likelihood
        posterior[2] <- numerat / denomin

        # visual - this is the third element in the vector
        numerat <- prior[3] * as.numeric(p_w_given_c$visual[p_w_given_c_index])
        denomin <- prior %*% vec_likelihood
        posterior[3] <- numerat / denomin

        if (sum(posterior)>1.01) {
          ERROR <- TRUE
        }

        prior <- posterior

      } # close loop around occurrences

    } # close loop around words in this review

  } # close if review has no sent words

  return (posterior_= posterior )
}
```

NBC Sentiment Analysis Loop

Now that we have defined the functions for calculating posteriors, we can loop over the reviews and apply these functions to determine the sentiment and content posteriors for each review. Using the 'Compute_posterior_sentiment' function we defined, we can calculate the posteriors for sentiment for each review using NBC.

```
# Loop over each review
for (review_index in 1:total_reviews) {

  # Print progress every 100th review
  if (review_index %% 100 == 0) {
    cat("Computing content of review #", review_index, " \n", sep="")
  }

  # If the review is not empty, continue and calculate posterior
  if ( reviews_raw$full_text[review_index] != ""){
```

```
    # Assign the processed text of the non-empty review to text_review
    text_review <- as.character(reviews_raw$processed_text[review_index])

    # Reset the prior every iteration as each review is looked at separately
    prior_sent_reset <- c(prior_sent, 1 - prior_sent)

    # Pre-process the review to remove punctuation marks and numbers.
    # Note that we are not removing stopwords here (nor elsewhere - a point for improvement)
    corpus_review <- tm_map(tm_map(VCorpus(VectorSource(text_review)), removePunctuation),
                            removeNumbers)

    # Compute posterior probability the review is positive
    TOT_DIMENSIONS <- 2
    sent.results <- Compute_posterior_sentiment(prior = prior_sent_reset,
                                                corpus_in = corpus_review,
                                                dict_words = lexicon_sentiment,
                                                p_w_given_c = likelihoods_sentim,
                                                TOT_DIMENSIONS)

    words_sent     <- sent.results$words_
    posterior_sent <- sent.results$posterior_

    reviews_raw$prob_sentiment[review_index] <- posterior_sent[1]
    reviews_raw$words_in_lexicon_sentiment_and_review[review_index] <- paste(words_sent,collapse =" ")

  }
}
```

NBC Content Analysis Loop

We also calculate the posteriors for the content each review using NBC.

```
# Loop over each review
for (review_index in 1: total_reviews) {

  # Print progress every 100th review
  if (review_index %% 100 == 0) {
    cat("Computing content of review #", review_index, " \n", sep="")
  }

  # If the review is not empty, continue and calculate posterior
  if ( reviews_raw$full_text[review_index]!=""){

    # Assign the processed text of the non-empty review to text_review
    text_review <- reviews_raw$processed_text[review_index]

    # Pre-process the review to remove numbers and punctuation marks.
    # Note that we are not removing stopwords here (nor elsewhere - a point for improvement)
    # put in corpus format and obtain word matrix
    corpus_review <- VCorpus(VectorSource(text_review))

    output <- capture.output(content_word_matrix <-
                               inspect(DocumentTermMatrix(corpus_review,
```

```
                                                    control = list(stemming=FALSE,
                                                                   language = "english",
                                                                   removePunctuation=TRUE,
                                                                   removeNumbers=TRUE,
                                                                   dictionary=as.character(lexicon

    # Compute posterior probability the review is about each topic
    TOT_DIMENSIONS <- 3
    posterior <- Compute_posterior_content(prior= matrix(prior_topic, ncol=TOT_DIMENSIONS),
                                           content_word_matrix,
                                           p_w_given_c=likelihoods,
                                           TOT_DIMENSIONS)

    # Store the posteriors
    reviews_raw$prob_storyline[review_index]    <- posterior[1]
    reviews_raw$prob_acting[review_index]       <- posterior[2]
    reviews_raw$prob_sound_visual[review_index] <- posterior[3]

  }
}

Processed_reviews <- reviews_raw
View(Processed_reviews)

# Saves the updated file, now including the sentiment and content/topic posteriors.
# write.csv(Processed_reviews,"TestProcessed_reviews.csv" , row.names = FALSE )
```

4. Supervised Learning: Syuzhet library and AFINN

AFINN is a sentiment analysis tool that uses a lexicon specifically designed to evaluate the sentiment score of texts. Each word in the lexicon is assigned a sentiment score from -5 to 5, indicating whether it is negative or positive, allowing us to score overall sentiments of texts based on the combined scores of individual words. Following a similar approach to the Naive Bayes Classifier, we calculate the sentiment for each review and add the results to our dataframe.

```
library(syuzhet)

# Loop over each review
for (review_index in 1:total_reviews) {

  # Print progress every 100th review
  if (review_index %% 100 == 0) {
    cat("Computing AFINN sentiment of review #", review_index, " \n", sep="")
  }

  # If the review is not empty, continue and apply AFINN
  if (reviews_raw$full_text[review_index] != ""){

    # Assign the processed text of the non-empty review to text_review
    # Note that we have not removed punctuation, numbers, and stopwords (a point for improvement)
    text_review <- as.character(reviews_raw$processed_text[review_index])

    # Apply AFINN
```

```
    AFINN <- get_sentiment(text_review, method = "afinn")

    # store the AFINN results in the dataframe
    reviews_raw$AFINN[review_index]      <- AFINN

  }
}

Processed_reviews <- reviews_raw
View(Processed_reviews)

#write.csv(Processed_reviews,"AFINN_Processed_reviews.csv" , row.names = FALSE )
```

5. Performance Measurement: Confusion matrix

A confusion matrix is a valuable tool for evaluating classification models. A confusion matrix helps us compare our model's predictions with the true values by summarizing the counts of correct and incorrect predictions across different classes. In classification problems, we typically focus on the positive class-the one we're interested in predicting-and the negative class, which represents all other outcomes. Below, you'll find an example of a confusion matrix.

A confusion matrix consists of: 1. True Positives (TP): Correctly predicted positive cases. 2. True Negatives (TN): Correctly predicted negative cases. 3. False Positives (FP): Incorrectly predicted as positive (Type I error). 4. False Negatives (FN): Incorrectly predicted as negative (Type II error).

Using a confusion matrix we can calculate metrics to calculate the performance of our classification model. A commonly used metric is the specificity The formula for the specificity is given below. A more comprehensive overview of metrics can be found here: https://en.wikipedia.org/wiki/Confusion_matrix#Table_of_confusion

Specificity = TN / (FP + TN)

Now imagine we have trained a model that predicts whether an incoming email into our inbox is spam or not. We could use such a model's predictions to evaluate its effectiveness. Below is an example of how such an evaluation might look like with artificial values for actual and predicted outcomes. So, these values are not based of off a real model, but are arbitrarily chosen to showcase how to use a confusion matrix in R and calculate specificity.

Artificial actual and predicted values for a classification problem with classes 'spam' and 'not spam'

```
actual <- factor(c("spam", "not spam", "spam", "spam", "not spam", "not spam", "spam", "not spam", "not
                   levels = c("spam", "not spam"))
predicted <- factor(c("spam", "not spam", "not spam", "spam", "not spam", "not spam", "spam", "spam", "
                      levels = c("spam", "not spam"))

# We can display the confusion matrix
conf_matrix <- confusionMatrix(predicted, actual)
conf_matrix$table

##           Reference
## Prediction spam not spam
##   spam        3        1
##   not spam    2        4

# Calculating the specificity
specificity(predicted, actual)
```

```
## [1] 0.8
```

You might have noticed that the supervised models in the previous sections do not output class labels (such as 'spam'/'not spam'), but rather a probability. At this stage, we distinguish between soft predictions and hard predictions. Soft predictions are the probabilities that an observation belongs to the positive class, while hard predictions are the final class labels assigned to the observations. To convert soft predictions into hard predictions, we use a decision rule, such as applying a threshold or selecting the class with the highest probability.

6. Brain teaser: hard vs. soft predictions (solutions)

A classifier gives each observation a probability for each of three content categories. Two ways to use it: keep the probabilities (soft), or round them to the winner (hard). Hard predictions are particularly useful when a ready-to-use prediction is required. However, making the hard prediction inherently loses the more detailed information on the predicted probabilities. For example, imagine that you are building an information databank for yourself at work, and would like to aggregate all the incoming emails that are useful to you. Here, the emails are first categorized as either useful or not useful. Then, if the email is judged as useful, the information contained in the email is passed to some storage unit. The problem becomes clear - here, you literally lose information if your classifier misjudges an email. Thus, it starts to make more sense to perhaps keep the probabilities on usefulness, so that you yourself can judge the email's content if the usefulness of its content is uncertain.

To showcase this problem, we construct the following scenario. We imagine 3 categories and a classifier that can make errors. We ask you to compare the information that results from using either hard or soft predictions.

6.1 Synthetic data: three category likelihoods + ground truth

Draw the likelihoods at random (a Dirichlet draw: three positive numbers normalised to sum to 1), then draw the true category from those very likelihoods. That is what a well-specified classifier means - when it says 0.7, the truth is that category 70% of the time.

```
set.seed(20250902)

n     <- 900                # number of observations
alpha <- c(1.0, 0.7, 0.5)   # category 1 is the frequent one, category 3 the rare one

# empty containers that we fill in observation by observation
p     <- matrix(0, nrow = n, ncol = 3)   # the three likelihoods of each observation
truth <- rep(0, n)                       # the true category of each observation

for (i in 1:n) {

  # draw three positive numbers and rescale them so that they sum to one
  draws <- c(rgamma(1, alpha[1]), rgamma(1, alpha[2]), rgamma(1, alpha[3]))
  probs <- draws / sum(draws)

  # store them, and draw the true category using those same probabilities
  p[i, ]   <- probs
  truth[i] <- sample(c(1, 2, 3), size = 1, prob = probs)
}

# the data we work with: three likelihood columns and the ground truth
```

```r
dat <- data.frame(p1 = p[, 1], p2 = p[, 2], p3 = p[, 3], truth = truth)
head(dat)
```

```
##          p1          p2          p3 truth
## 1 0.7345365 0.008264207 0.257199272     3
## 2 0.5115798 0.350565996 0.137854239     1
## 3 0.5115089 0.212786995 0.275704105     1
## 4 0.1747086 0.438967167 0.386324271     2
## 5 0.4034143 0.296779242 0.299806438     1
## 6 0.8860766 0.107666054 0.006257377     2
```

```r
# average likelihood per category, and the share of each category in the truth
round(colMeans(p), 3)
```

```
## [1] 0.457 0.307 0.236
```

```r
round(table(truth) / n, 3)
```

```
## truth
##     1     2     3
## 0.466 0.310 0.224
```

6.2 Hard predictions: argmax

```r
# the hard prediction is simply the category with the highest likelihood
pred    <- rep(0, n)   # the predicted category
winning <- rep(0, n)   # the probability that won, i.e. how sure the classifier was

for (i in 1:n) {
  pred[i]    <- which.max(p[i, ])
  winning[i] <- max(p[i, ])
}

# confusion matrix: the truth in the rows, the prediction in the columns
table(truth = truth, predicted = pred)
```

```
##      predicted
## truth   1   2   3
##     1 313  64  42
##     2  81 163  35
##     3  57  31 114
```

```r
acc <- mean(pred == truth)
round(acc, 3)                          # accuracy
```

```
## [1] 0.656
```

```r
round(max(table(truth) / n), 3)        # majority baseline
```

```
## [1] 0.466
```

```r
round(mean(winning), 3)                # average winning probability
```

```
## [1] 0.662
```

Accuracy is about 66%, against a majority baseline of about 47%. That is close to the ceiling:

the average winning probability is about 0.66, so no rule that has to commit to one category can do much better on these data. Note where the errors sit - categories 2 and 3 lose far more of their observations than

category 1, because a rare category rarely wins a comparison even when it is the true one.

6.3 Soft predictions: keep the probabilities

When evaluating forecasts of probabilities, data scientists often make use of the Brier score. In our case, we score the probabilities constructed above with the multiclass Brier score, (1/n) * sum_i sum_k (p_ik - y_ik)^2, where y is 1 if the prediction is correct. This is especially appropriate when evaluating our soft predictions, which unlike the hard predictions, take on continuous values.

```
# the three forecasts we compare, each one a table with a probability per category

# 1. the soft probabilities: the matrix p we already have

# 2. the hard prediction written as probabilities: all the mass on the winner
p_hard <- matrix(0, nrow = n, ncol = 3)
for (i in 1:n) {
  p_hard[i, pred[i]] <- 1
}

# 3. the baseline: the same average likelihoods for every observation
p_base <- matrix(0, nrow = n, ncol = 3)
for (i in 1:n) {
  p_base[i, ] <- colMeans(p)
}

# the Brier score: per observation, the squared distance between the forecast
# and the truth written as 0/1, then averaged over all observations
brier <- function(forecast) {

  scores <- rep(0, n)

  for (i in 1:n) {
    y           <- c(0, 0, 0)      # the truth of observation i as 0/1
    y[truth[i]] <- 1
    scores[i]   <- sum((forecast[i, ] - y)^2)
  }

  mean(scores)
}

results <- data.frame(
  forecast = c("Soft probabilities", "Hard 0/1 (argmax)", "Average likelihood"),
  brier    = c(brier(p), brier(p_hard), brier(p_base))
)
results
```

```
##             forecast     brier
## 1 Soft probabilities 0.4471268
## 2  Hard 0/1 (argmax) 0.6888889
## 3 Average likelihood 0.6370110
```

```
# Aggregate use: what share of the observations belongs to category 1?
round(mean(truth == 1), 3)   # the true share
```

```
## [1] 0.466
```

```r
round(mean(p[, 1]), 3)        # estimated with the soft probabilities
```

```
## [1] 0.457
```

```r
round(mean(pred == 1), 3)     # estimated by counting the hard predictions
```

```
## [1] 0.501
```

The rounded forecast is built from the same information, yet its Brier score is about 1.5 times

worse than that of the probabilities it came from - the price of claiming certainty on observations where the model was near 0.5. A confident wrong call costs more than an honest vague one.

6.4 Takeaway

Argmax answers "which category?" and is the right output when one decision must be made per observation. It is the wrong output when the classification is an intermediate step feeding an average or a regression: the probabilities score better and aggregate without bias, which is why the review data ship prob_storyline, prob_acting and prob_sound_visual rather than one label per sentence.